TP Continuous Integration

Industrialisation CI d'une application React + Node.js avec GitHub Actions, Docker et SonarQube

Mohamed Najeh ISSAOUI
E-mail : issaoui.mn@itbs.tn
IT Business School

1. Contexte du TP

Jusqu'à présent, vous avez appris à :

- développer une application localement avec **VS Code**
- organiser votre projet en **frontend React** et **backend Node.js**
- créer des fichiers **Dockerfile**
- utiliser **docker-compose**
- construire des images Docker localement
- utiliser **Git** et pousser votre code vers un dépôt **GitHub**
- expérimenter localement **Prometheus**, **Grafana** et **Kubernetes avec Minikube**

À ce stade, votre application fonctionne **localement**.

Dans ce TP, nous allons franchir une nouvelle étape fondamentale en DevOps :

Automatiser la vérification de votre projet à chaque push sur GitHub

Autrement dit, au lieu de faire manuellement :

- le lint
- les tests
- l'analyse qualité
- le build Docker

Vous allez demander à **GitHub Actions** de le faire automatiquement.

2. Objectif pédagogique global

L'objectif de ce TP est de vous faire comprendre et mettre en œuvre la **CI — Continuous Integration (Intégration Continue)**.

À la fin du TP, vous devrez être capables de :

- expliquer ce qu'est la CI
- expliquer le rôle de **GitHub Actions**
- créer un fichier **.github/workflows/ci.yml**
- automatiser le **lint**
- automatiser les **tests**
- intégrer **SonarQube / SonarCloud**
- construire les **images Docker** dans le pipeline
- comprendre l'usage des **Secrets GitHub**
- analyser le résultat d'un pipeline dans GitHub

3. Rappel du projet de départ

On suppose que votre dépôt GitHub contient un projet structuré ainsi :

```
mon-projet/
│
├── frontend/
│   ├── src/
│   ├── public/
│   ├── package.json
│   ├── Dockerfile
│   └── ...
│
├── backend/
│   ├── src/
│   ├── package.json
│   ├── Dockerfile
│   └── ...
│
├── docker-compose.yml
└── README.md
```

- `frontend/` : application **React**
- `backend/` : application **Node.js / Express**

4. Partie Cours intégrée — notions à comprendre avant l'implémentation

4.1 Qu'est-ce que la CI ?

La **CI (*Continuous Integration*)** est une pratique DevOps qui consiste à :

Vérifier automatiquement le projet à chaque changement de code

Lorsqu'un développeur fait :

```
git push
```

Un pipeline automatique peut s'exécuter pour :

- récupérer le code

- installer les dépendances
- analyser la qualité du code
- lancer les tests
- construire l'application
- construire les images Docker

Pourquoi la CI est-elle importante ?

Parce qu'elle permet de :

- détecter les erreurs tôt
- éviter de fusionner du code cassé
- standardiser la qualité
- gagner du temps
- préparer le projet à la suite du pipeline DevOps

4.2 Qu'est-ce que GitHub Actions ?

GitHub Actions est le système d'automatisation intégré à GitHub.

Il permet d'écrire un scénario automatique dans un fichier YAML qui dit à GitHub :

- quand lancer le pipeline
- quelles étapes exécuter
- dans quel ordre
- avec quels outils

Le fichier se place ici :

```
.github/workflows/ci.yml
```

4.3 Qu'est-ce que le lint ?

Le **lint** est une analyse automatique du code source.

Il permet de détecter :

- erreurs simples
- incohérences de style
- mauvaises pratiques
- oublis fréquents

Important

Le lint **ne teste pas** le comportement du programme.

Il vérifie surtout :

La qualité d'écriture du code

Outil utilisé dans ce TP

Pour le frontend React et le backend Node.js, nous utiliserons :

- **ESLint**

4.4 Qu'est-ce qu'un test dans ce TP ?

Un test automatique vérifie qu'une partie du programme fonctionne comme attendu.

Dans ce TP, on peut rencontrer :

Tests unitaires

Ils testent une fonction ou un composant isolé.

Tests backend

Ils testent une route, une fonction, ou une logique métier.

Très important

Dans le pipeline GitHub Actions, les tests doivent être :

- automatiques
- non interactifs
- reproductibles

Nous n'utiliserons pas ici de test manuel dans la CI.

4.5 Qu'est-ce que SonarQube ?

SonarQube est un outil d'analyse de qualité de code.

Il détecte notamment :

- bugs potentiels
- duplications
- complexité excessive
- vulnérabilités potentielles
- dette technique

Attention importante

Quand le pipeline tourne dans GitHub Actions, il s'exécute sur une machine distante appartenant à GitHub.

Donc :

- Si votre serveur SonarQube est local sur votre PC, **GitHub Actions ne pourra pas le voir**
- Il faut soit :
  - utiliser **SonarCloud**
  - soit utiliser un **serveur SonarQube accessible depuis Internet**

→ Pour un TP pédagogique, la solution la plus simple est souvent **SonarCloud**.

5. Fichiers et outils nécessaires avant de commencer

5.1 Où créer les fichiers ?

Vous avez **deux possibilités** :

Option A — Recommandée : créer/modifier les fichiers dans VS Code

C'est la méthode la plus propre si vous travaillez déjà localement.

Étapes :

1. ouvrir le projet dans **VS Code**
2. créer ou modifier les fichiers
3. sauvegarder
4. faire :

```
git add .
git commit -m "ajout pipeline CI"
git push
```

Option B — Depuis l'interface GitHub

GitHub permet aussi de créer directement un fichier dans le navigateur.

Exemple pour créer `ci.yml` :

- ouvrir le repository GitHub
- cliquer sur **Add file**
- puis **Create new file**
- écrire le chemin :

```
.github/workflows/ci.yml
```

- coller le contenu
- commit direct

→ Pour ce TP, **VS Code est recommandé**, car il permet de mieux comprendre la structure du projet.

5.2 Préparer les scripts package.json

Avant la CI, vos projets doivent déjà savoir exécuter localement :

- le lint
- les tests
- le build

Frontend React — frontend/package.json

```
{
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test --watchAll=false",
    "lint": "eslint src --ext .js,.jsx"
  }
}
```

Explication

- `start` : lance l'application React
- `build` : construit la version de production
- `test` : lance les tests sans mode interactif
- `lint` : vérifie la qualité du code React

Backend Node — backend/package.json

```
{
  "scripts": {
    "start": "node src/index.js",
    "test": "jest",
    "lint": "eslint src --ext .js"
  }
}
```

6. Étape 1 — Vérifier localement avant d'automatiser

Avant d'écrire le pipeline, il faut que le projet fonctionne **déjà localement**.

Frontend

```
cd frontend
npm install
npm run lint
```

```
npm test
npm run build
```

Backend

```
cd backend
npm install
npm run lint
npm test
```

Pourquoi cette étape est-elle obligatoire ?

Parce que le pipeline GitHub Actions ne doit pas être utilisé pour “deviner” ce qui manque dans le projet.

La CI sert à **automatiser une procédure déjà connue et validée localement**.

7. Étape 2 — Préparer SonarQube / SonarCloud

7.1 D’où vient `SONAR_TOKEN` ?

Dans un pipeline GitHub Actions, on écrit souvent :

```
env:
  SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
```

Explication

`secrets.SONAR_TOKEN` signifie :

GitHub va lire une valeur secrète enregistrée dans les **Secrets du repository**

7.2 Pourquoi ne pas écrire le token directement dans `ci.yml` ?

Parce qu’un token est une donnée sensible à ne pas partager

Exemple à ne jamais faire :

```
env:
  SONAR_TOKEN: abc123456789
```

C’est dangereux !

7.3 Comment obtenir `SONAR_TOKEN` ?

Cas le plus simple : SonarCloud

1. créer un compte SonarCloud

2. se connecter
3. aller dans :
    - **My Account**
    - **Security**
4. générer un token
5. copier ce token

7.4 Où mettre le token dans GitHub ?

Dans votre repository GitHub :

- aller dans **Settings**
- puis **Secrets and variables**
- puis **Actions**
- cliquer sur **New repository secret**

Créer un secret avec :

- **Name** : `SONAR_TOKEN`
- **Secret** : coller le token généré dans SonarCloud

→ Ensuite GitHub pourra utiliser ce secret dans le pipeline avec :

```
${{ secrets.SONAR_TOKEN }}
```

7.5 D'où vient `sonar.projectKey` ?

Exemple :

```
sonar.projectKey=mon-projet-react-node
```

Cet identifiant correspond au **nom technique du projet dans SonarQube / SonarCloud**.

7.6 Comment obtenir `sonar.projectKey` ?

Quand vous créez un projet dans SonarCloud ou SonarQube :

1. vous cliquez sur **Create project**
2. vous choisissez un nom
3. l'outil vous attribue ou vous demande un identifiant

Exemple :

- nom visible : `Mon Projet React Node`
- clé technique : `mon-projet-react-node`

Cette clé sera utilisée par le scanner pour dire :

“voici le projet Sonar que je dois mettre à jour”

7.7 Que faut-il créer comme fichier de configuration Sonar ?

À la racine du dépôt, créer :

```
sonar-project.properties
```

Ce fichier peut être créé :

- dans **VS Code**
- ou dans GitHub via **Add file**

7.8 Exemple de `sonar-project.properties`

```
sonar.projectKey=mon-projet-react-node
sonar.projectName=Mon Projet React Node
sonar.sources=frontend/src,backend/src
sonar.tests=frontend/src,backend/src
sonar.javascript.lcov.reportPaths=frontend/coverage/lcov.info,backend/cover
age/lcov.info
sonar.sourceEncoding=UTF-8
```

7.9 Explication ligne par ligne

`sonar.projectKey` → Identifiant technique du projet dans SonarQube / SonarCloud.

`sonar.projectName` → Nom lisible affiché dans l’interface.

`sonar.sources` → Répertoires du code source à analyser.

`sonar.tests` → Répertoires contenant du code de tests.

`sonar.javascript.lcov.reportPaths` → Chemins des rapports de couverture générés par les tests.

`sonar.sourceEncoding` → Encodage du projet.

8. Étape 3 — Créer le pipeline GitHub Actions

Créer le fichier :

```
.github/workflows/ci.yml
```

→ Encore une fois, le plus simple est de le faire dans **VS Code**.

9. Fichier `ci.yml` complet — version avec des commentaires

```
name: CI React Node

on:
  push:
    branches:
      - main
      - develop
  pull_request:
    branches:
      - main
      - develop

jobs:
  frontend:
    runs-on: ubuntu-latest

    steps:
      - name: 1. Récupérer le code du dépôt
        uses: actions/checkout@v4

      - name: 2. Installer Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 18

      - name: 3. Mettre en cache les dépendances npm du frontend
        uses: actions/cache@v3
        with:
          path: ~/.npm
          key: ${{ runner.os }}-frontend-${{ hashFiles('frontend/package-
lock.json') }}
          restore-keys: |
            ${{ runner.os }}-frontend-

      - name: 4. Installer les dépendances du frontend
        run: |
          cd frontend
          npm ci

      - name: 5. Lancer ESLint sur le frontend
        run: |
          cd frontend
          npm run lint

      - name: 6. Exécuter les tests frontend avec couverture
        run: |
          cd frontend
          npm test -- --watchAll=false --coverage

      - name: 7. Construire l'application React
        run: |
          cd frontend
          npm run build

      - name: 8. Construire l'image Docker du frontend
        run: |
```

```
          docker build -t frontend-app:latest ./frontend

  backend:
    runs-on: ubuntu-latest

    steps:
      - name: 1. Récupérer le code du dépôt
        uses: actions/checkout@v4

      - name: 2. Installer Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 18

      - name: 3. Mettre en cache les dépendances npm du backend
        uses: actions/cache@v3
        with:
          path: ~/.npm
          key: ${{ runner.os }}-backend-${{ hashFiles('backend/package-
lock.json') }}
          restore-keys: |
            ${{ runner.os }}-backend-

      - name: 4. Installer les dépendances du backend
        run: |
          cd backend
          npm ci

      - name: 5. Lancer ESLint sur le backend
        run: |
          cd backend
          npm run lint

      - name: 6. Exécuter les tests backend avec couverture
        run: |
          cd backend
          npm test -- --coverage

      - name: 7. Analyser la qualité du backend avec SonarQube / SonarCloud
        run: |
          npx sonar-scanner
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}

      - name: 8. Construire l'image Docker du backend
        run: |
          docker build -t backend-app:latest ./backend
```

10. Explication pédagogique détaillée des étapes du pipeline

10.1 Bloc on :

```
on:
  push:
    branches:
      - main
      - develop
```

```
  pull_request:
    branches:
      - main
      - develop
```

Rôle

Ce bloc indique **quand** la CI doit s'exécuter.

Interprétation

- si on pousse sur `main` ou `develop`, le pipeline démarre
- si on crée ou met à jour une Pull Request vers `main` ou `develop`, le pipeline démarre aussi

10.2 Bloc `jobs` :

Il contient deux jobs :

- frontend
- backend

Cela permet de traiter séparément les deux parties du projet.

10.3 `actions/checkout@v4`

```
uses: actions/checkout@v4
```

Rôle

Télécharge le code du dépôt dans la machine GitHub Actions.

Sans cela, le pipeline n'a pas accès au projet.

10.4 `actions/setup-node@v4`

```
uses: actions/setup-node@v4
with:
  node-version: 18
```

Rôle

Installe Node.js dans la machine distante GitHub Actions.

Pourquoi c'est important ?

Parce qu'on veut un environnement stable, reproductible et identique pour tous.

10.5 Cache npm

```
uses: actions/cache@v3
with:
  path: ~/.npm
  key: ${{ runner.os }}-frontend-${{ hashFiles('frontend/package-
lock.json') }}
```

Rôle

Mémoriser les dépendances npm déjà téléchargées.

Pourquoi ?

Pour accélérer les exécutions suivantes du pipeline.

Idée importante

Si le `package-lock.json` change, le cache doit changer aussi.

10.6 `npm ci`

```
run: |
  cd frontend
  npm ci
```

Pourquoi `npm ci` ?

Parce que c'est la commande recommandée en CI :

- rapide
- reproductible
- basée sur le lockfile

10.7 Lint

```
run: |
  cd frontend
  npm run lint
```

Rôle

Vérifier que le code respecte les règles définies par ESLint.

Si erreur :

le pipeline s'arrête.

10.8 Tests

```
run: |
  cd frontend
  npm test -- --watchAll=false --coverage
```

Explication

- `--watchAll=false` : mode non interactif
- `--coverage` : génère un rapport de couverture

10.9 SonarQube / SonarCloud

```
run: |
  npx sonar-scanner
env:
  SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
```

Rôle

Lancer le scanner Sonar.

Très important

`SONAR_TOKEN` ne vient pas du code, mais des **Secrets GitHub**.

10.10 Docker build

```
run: |
  docker build -t backend-app:latest ./backend
```

Rôle

Construire l'image Docker dans le pipeline GitHub.

Pourquoi cette étape est importante ?

Elle montre que le projet est :

- dockerisable
- reproductible
- prêt pour la suite du pipeline DevOps

11. Étape 4 — Push du pipeline vers GitHub

Quand le fichier `ci.yml` est prêt :

```
git add .
git commit -m "ajout pipeline CI React Node"
git push
```

À ce moment-là :

- GitHub détecte le fichier
- le workflow apparaît dans l'onglet **Actions**
- le pipeline s'exécute automatiquement

12. Qu'est-ce que vous observez ?

Dans l'onglet **Actions** :

- le workflow `CI React Node`
- les jobs :
  - `frontend`
  - `backend`
- les étapes de chaque job
- les erreurs éventuelles

Dans SonarQube / SonarCloud :

- le projet analysé
- les problèmes détectés
- le tableau de bord qualité

13. A retenir à la fin

À la fin de ce TP, l'étudiant doit comprendre que :

- GitHub n'est pas seulement un dépôt de code
- GitHub Actions transforme GitHub en plateforme d'automatisation
- la qualité du code peut être contrôlée automatiquement
- la CI exécute les mêmes contrôles que le développeur, mais dans un environnement standard
- Docker dans la CI permet de certifier un artefact reproductible
- SonarQube ajoute une couche d'analyse qualité approfondie